

Script Composer Evolution — v1.0 Architecture

Status: Adopted · **Version:** 1.0 · **Owner:** Prasanth (LevelUp AI QA)

Applies to: `levelup-ai-qa-agent` (backend script generation) and its dashboard consumer.

This document is the reference for every Script Composer PR going forward. Cite it in each PR.

0. Purpose

The scenario pipeline is done. Phases A/B (Scenario Data Model + Renderers — PR #247) gave us a single canonical scenario, projected per consumer. We are **not** going to keep redesigning the pipeline.

From here, we make the **existing Script Composer progressively smarter** through deterministic engineering heuristics, ranking algorithms, and quality refinements — never through new orchestrators, intelligence providers, writers, planners, or pipelines.

This document freezes the architecture at **v1.0** and defines the incremental roadmap (Sprints 1–7 + the Engineering Heuristics Library) that raises generated-code quality without destabilizing the foundation.

1. The Freeze Mandate (read this first)

Freeze the architecture. Do not introduce new orchestrators, intelligence providers, writers, planners, or pipelines. All future improvements must occur inside the existing Script Composer through deterministic engineering heuristics, ranking algorithms, and quality refinements. Every PR must make the generated framework look more like code written by an experienced automation engineer while preserving existing capabilities and avoiding regressions.

The single measure of success

Every future PR is judged by one question:

“Would a senior Playwright engineer approve this pull request without realizing it was AI-generated?”

If a change does not move us toward **yes**, it does not belong in this roadmap.

What this explicitly forbids

- **✗** New orchestrators / intelligence providers / writers / planners / pipelines
- **✗** Moving responsibilities between frozen components (see §2)

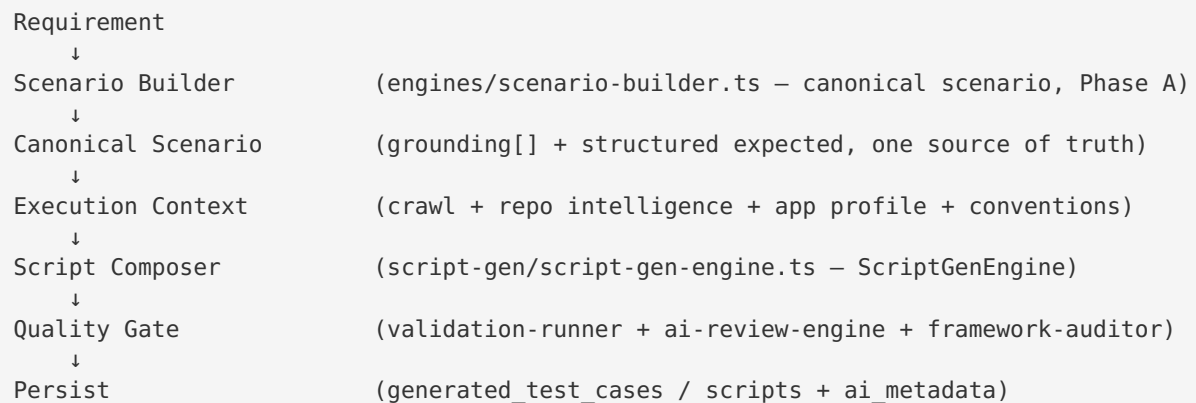
- ❌ More comments, AI explaining every action, verbose logging
- ❌ Huge helper methods, generated documentation, over-engineering
- ❌ Defaulting to the LLM when a deterministic heuristic can decide

What this encourages

- ✅ Better **decisions** inside existing engines (ranking, scoring, fallback)
- ✅ Deterministic heuristics over prompts (no token growth)
- ✅ Reuse of existing framework assets over regeneration
- ✅ Honest confidence signals over pretended certainty

2. The Frozen Pipeline (v1.0)

These are foundation components. **Freeze them. Do not keep moving responsibilities around.**



Where each frozen stage lives in the codebase

Stage	Primary module(s)
Scenario Builder	<code>src/engines/scenario-builder.ts</code>
Canonical Scenario	<code>DraftTestCase / FormatterTestCase</code> (<code>grounding[]</code> , <code>structured expected</code> , <code>schemaVersion</code>)
Renderers / Projection	<code>src/renderers/scenario-renderer.ts</code> (Manual / Script / BDD)
Execution Context	<code>src/script-gen/page-crawler.ts</code> , <code>src/context/*</code> , <code>src/intelligence/project-convention-profile.ts</code>
Script Composer	<code>src/script-gen/script-gen-engine.ts</code> (<code>ScriptGenEngine</code>)
Quality Gate	<code>src/script-gen/validation-runner.ts</code> , <code>ai-review-engine.ts</code> , <code>framework-auditor.ts</code>
Persist	<code>src/api/routes/script-gen.ts</code> , <code>test-coverage.ts</code>

Everything in the roadmap below happens **inside the Script Composer** and its owned sub-engines. No stage boundary moves.

3. Evolve the Script Composer — not the pipeline

The Script Composer (`ScriptGenEngine`) already composes the sub-engines we will sharpen. We are **not** adding engines; we are making the existing ones decide better.

Sub-engine (exists today)	Role	Sprint that sharpens it
<code>SelectorQualityEngine + intelligence/element-intelligence.rankLocatorCandidates</code>	Locator scoring	Sprint 2
<code>AssertionEngine</code>	Assertions	Sprints 3 & 4
<code>project-convention-profile (buildReuseCatalogue , find-ReusablePageObject)</code>	Reuse	Sprint 5
<code>ai-review-engine / framework-auditor</code>	Style / audit	Sprint 6
<code>engines/confidence-engine.ts</code>	Confidence	Sprint 7
<code>core/candidate-ranker.ts + core/advisors/*</code>	Ranking (healing side)	Reuse target for Sprint 2

Reuse-first note: the healing side already has a mature scored candidate ranker (`core/candidate-ranker.ts` with `SOURCE_TRUST` , plus `core/advisors/{rule-engine,app-profile,dom-candidate,dom-memory,learned-pattern,ai}-advisor.ts`). Sprint 2 should **reuse and extend** this ranking substrate for generation, not build a parallel one. One ranking brain, two callers.

4. Roadmap

Sprint 1 — Fix & Freeze (this PR set)

-  Fix the **Generate Script** regression (Test Case Lab → Script Gen deep link now selects the requirement, prefills the scenario, and hides/omits App Knowledge because the test case is ready). (dashboard PR #146)
-  Merge PR #247 (Scenario Data Model + Renderers). (owner action)
-  Create this document (`SCRIPT_COMPOSER_EVOLUTION.md`) and **declare the architecture frozen**.
- **No new intelligence.**

Sprint 2 — Locator Ranking Engine (highest ROI)

This is **not** “better locators.” It is **better locator decision-making**.

Today

Grounding → Selector

Target

Grounding → Candidate Discovery → Candidate Ranking → Candidate Selection → Confidence

4.1 Candidate Discovery — collect, do not select

Gather candidates from every source, in priority order. **Do not choose yet.**

1. Existing **Page Object** methods (reuse wins)
2. **App Profile** (crawled grounding)
3. **Test Case** wording
4. **Expected Result** text
5. **Requirement** text
6. **DOM** relationships
7. **Accessibility** (role + name, aria-label, label association)
8. **AI** — last resort only

Fallback search chain before AI. When the App Profile misses an element, do **not** jump to the LLM. Search deterministically first:

Test Case → Keywords → Expected Result → Requirement → Page Object → DOM → AI

Example — manual step Click Login, expected Dashboard appears : the composer infers Login causes navigation and searches for login / sign in / submit / authenticate candidates instead of hallucinating a selector.

4.2 Candidate Ranking — score every candidate

Reuse/extend `core/candidate-ranker.ts` scoring. Indicative scores:

Source / strategy	Score
Existing Page Object method	100
<code>data-testid</code>	98
<code>role</code> + accessible name	96
<code>aria-label</code>	95
label association	94
placeholder	88
stable CSS	82
DOM fingerprint	78
XPath	40

Never just use the first locator. Find → Rank → Choose.

4.3 Candidate Selection

- Choose the highest-scoring candidate.

- If the top two are close, keep **both** — record the runner-up as an alternative:

Chosen: `getByRole('button', { name: 'Login' })`

Alternative: `getByTestId('login-btn')`

4.4 Confidence banding (feeds Sprint 7)

Confidence	Behavior
> 90	Generate normally.
70-90	Generate normally and record Review recommended.
< 70	Generate a <code>TODO</code> with a reason (e.g. Dynamic element) and a suggested locator.

Milestones (ship small — measure after each):

- **2a** — Candidate Discovery: collect candidates from all sources into a typed list (no behavior change to selection yet).
- **2b** — Ranking: score candidates via the shared ranker; log chosen vs. alternatives.
- **2c** — Selection + alternative retention.
- **2d** — Confidence banding + TODO emission for `< 70`.

Sprint 3 — Assertion Expansion ★★★★★

Automation should validate **more** than the manual test, because it can. Deterministic expansion inside `AssertionEngine` — **not** prompting.

Input `Verify login successful` → **Output**

- ☒ URL changed
- ☒ Dashboard visible
- ☒ Logout button visible
- ☒ Session established
- ☒ No error present
- ☒ Inventory / landing content loaded

Rules: verify the **business outcome first**, then add one or two **stability** assertions. Never generate redundant assertions.

Sprint 4 — Coverage Expansion ★★★★★

Different from Sprint 3. Not more assertions — **more business coverage**.

Manual `Add item to cart` → **Automation additionally verifies**

- cart badge updated
- item count
- total price
- persistence (survives reload / navigation)

- navigation correctness

Meaningful, business-relevant checks only — never random assertions. Automation coverage should **always exceed** the manual test case.

Sprint 5 — Framework Reuse ★★★★★

Improve **reuse**, not generation. Aggressively consult

`project-convention-profile ( buildReuseCatalogue , findReusablePageObject )` and repo pattern analysis.

```
login() ; login() ; login()    →    authenticatedFixture()
```

If `login()` (or any page-object method / fixture / helper) already exists, **reuse it — never regenerate**. The less code we generate, the more senior it looks.

Sprint 6 — Humanization ★★★★★

The framework must not feel generated. The problem is rarely correctness — it is that the output is **too symmetrical**. Senior engineers extract methods, simplify, and drop noise.

Reduce, inside `ai-review-engine / framework-auditor`:

- repetition
- unnecessary comments
- unnecessary variables
- over-defensive code
- identical naming everywhere

Make it feel like a teammate wrote it — following project conventions.

Sprint 7 — Confidence Engine ★★★★★

Last. Do not rewrite. Do not regenerate. Simply identify uncertainty and be honest about it, building on `engines/confidence-engine.ts` and the Sprint 2 bands.

Before returning, the composer asks: **“Would I merge this PR?”** — checking locator, assertion, reuse, compile, and framework confidence. When confidence is low, **flag**, don’t guess.

Smart TODOs (not AI noise)

Emit TODOs **only** when confidence is genuinely low — never a wall of `TODO TODO TODO`.

```
// Review locator:
// Dynamic element detected after login.
// Suggested locator:
// page.getByRole('button', { name: /checkout/i })
```

5. The Engineering Heuristics Library (the secret sauce)

Not another intelligence. A **deterministic rule library** that captures how experienced automation engineers think. Every generation consults it. **No LLM. No prompt. No tokens.** It improves the product over time without growing prompt size.

Proposed home: `src/script-gen/heuristics/` (a pure, dependency-light rule module consumed by `ScriptGenEngine`). Rules are data + pure functions — unit-tested and versioned.

Rule families

Locator heuristics

- Prefer existing page-object methods.
- Prefer `data-testid`.
- Prefer accessible roles and names.
- Avoid brittle CSS / XPath unless necessary.

Assertion heuristics

- Verify business outcome first.
- Add one or two stability assertions.
- Don't generate redundant assertions.

Framework heuristics

- Reuse existing fixtures.
- Reuse page-object methods.
- Don't create duplicate utilities.

Naming heuristics

- Follow the project's existing naming conventions.
- Avoid identical/templated names everywhere.

Playwright heuristics

- No `waitForTimeout`; use web-first assertions and auto-waiting.
- Prefer user-facing locators (`getByRole` / `getByLabel`).

These heuristics evolve over time (Sprint 7+ “continuous learning” improves the weights from successful generations) — **without** increasing token usage or architectural complexity.

6. Long-term stable roadmap

Phase	Goal	Change type
1	Fix & Freeze	Regression fix + freeze declaration
2	Locator Ranking Engine	Improve selection , no architecture changes
3	Assertion Expansion	Increase automation coverage
4	Coverage Expansion	Add meaningful automation-only validations
5	Framework Reuse	Reduce duplication
6	Human-like Code Style	Output indistinguishable from hand-written code
7	Confidence Engine	Flag uncertain areas instead of guessing
(ongoing)	Continuous Learning	Improve heuristic weights from successful generations

Notice what is missing: no new orchestrators, no new intelligence providers, no new pipelines, no major rewrites.

7. PR checklist (paste into every Script Composer PR)

- [] Change lives **inside** the Script Composer / its owned sub-engines — no new orchestrator/provider/writer/planner/pipeline, no stage boundary moved.
- [] Prefers a **deterministic heuristic** over an LLM call where a decision can be made.
- [] Reuses existing framework assets (page objects, fixtures, helpers) before generating.
- [] Adds **no** unnecessary comments / logging / variables / defensive code.
- [] Preserves existing capabilities (no regressions); tests updated.
- [] Emits honest confidence / Smart TODOs instead of pretending certainty.
- [] Answers **yes** to: “Would a senior Playwright engineer merge this without realizing it was AI-generated?”
- [] References this document (`SCRIPT_COMPOSER_EVOLUTION.md`).

8. Appendix — component map (as of v1.0)

src/engines/scenario-builder.ts	canonical scenario (Phase A)
src/renderers/scenario-renderer.ts	Manual / Script / BDD projection (Phase B)
src/script-gen/script-gen-engine.ts	ScriptGenEngine – the Script Composer
├─ selector-quality-engine.ts	locator scoring → Sprint 2
├─ assertion-engine.ts	assertions → Sprints 3 & 4
├─ wait-strategy-engine.ts	smart waits
├─ workflow-mapper.ts	navigation graph & flows
├─ validation-runner.ts	compile / validate (Quality Gate)
├─ ai-review-engine.ts	style review → Sprint 6
└─ framework-auditor.ts	framework audit → Sprint 6
src/intelligence/element-intelligence.ts	rankLocatorCandidates → Sprint 2
src/intelligence/project-convention-profile.ts	reuse catalogue → Sprint 5
src/core/candidate-ranker.ts	scored ranker (reuse) → Sprint 2
src/core/advisors/*	ranking advisors (reuse) → Sprint 2
src/engines/confidence-engine.ts	confidence → Sprint 7
src/script-gen/heuristics/ (proposed)	Engineering Heuristics Library → §5